The Open Source Audit

GIT

Version Control · GPL v2 · The Tool Linus Built When Proprietary Failed Him

Student Name	Registration Number
Shruti Deshmukh	24BAI10552
Slot	**Date of Submission**
A24	30 March 2026

Table of Contents

Part A — Origin and Philosophy

A1. The Problem Git Was Created to Solve

A2. The License — What It Actually Says

A3. The Ethics of Open Source

Part B — Linux Footprint

Installation, Directories, Users, Services, Updates

Part C — The FOSS Ecosystem

Dependencies, Forks, Web Relation, Community & Governance

Part D — Open Source vs Proprietary

Comparison Table & Verdict

Part E — Shell Scripts

Script1. System Identity Report

Script2. Package Inspector

Script3. Disk Auditor

Script4. Log Analyzer

Script5. Manifest Generator

Part A - Origin and Philosophy

A1. The Problem Git Was Created to Solve

Git was created in 2005 by Linus Torvalds after the Linux kernel project lost access to BitKeeper, a proprietary version control system. This highlighted the risk of relying on closed-source tools.

Earlier systems like CVS and SVN were centralized, requiring a constant server connection and making branching difficult. They were slow and lacked strong data integrity.

Git solved these issues by introducing a distributed model where every developer has a full copy of the repository. It offers high speed, reliable branching, and secure data handling through hashing. The Linux kernel quickly adopted it, and Junio Hamano later became its maintainer.

Today, Git is the global standard for version control and supports platforms like GitHub and GitLab.

A2. The License — What It Actually Says

Git is licensed under GPL v2, ensuring it remains free and open. According to Richard Stallman, free software provides four freedoms: use, study, modify, and share.

GPL enforces *copyleft*, meaning any distributed modifications must also be open source. However, internal use does not require sharing changes, which led to the Affero GPL.

Compared to MIT (a permissive license), GPL prioritizes openness. Git is both free of cost and free in terms of user control.

A3. The Ethics of Open Source

Open source improves transparency, collaboration, and independence, especially in critical systems. It also prevents vendor lock-in.

However, development requires funding, and many projects face sustainability issues. Companies may need proprietary models to protect innovation.

Organizations like Google and Meta rely on open source, raising ethical questions. While commercial use is allowed, companies should contribute back. Incidents like Log4j show risks when critical projects are underfunded.

Overall, open source enables innovation but requires shared responsibility to maintain it.

Part B — Linux Footprint

This section explains how Git operates on a Linux system, including installation, file locations, permissions, and management. All commands were tested on Ubuntu 22.04 LTS.

Installation Method

Git is available in all major Linux distributions and can be installed using package managers:

- Ubuntu/Debian: sudo apt install git
- Fedora/RHEL: sudo dnf install git
- Arch Linux: sudo pacman -S git

For newer versions, Git can be compiled from source using build tools like gcc, make, and required libraries.

Key Directories

Git follows the standard Linux directory structure:

Directory	Contents
/usr/bin/git	The main Git binary executable
/usr/libexec/git-core/	Sub-commands (git-add, git-commit, etc.)
~/.gitconfig	Per-user global configuration (user name, email, aliases)
/etc/gitconfig	System-wide configuration applied to all users
.git/ (per repo)	Repository data: objects, refs, config, HEAD, hooks

These directories store configuration, documentation, and version history data.

User and Group (Security)

Git runs under the current user for normal operations, ensuring it only has the permissions of that user.

For shared repositories, a dedicated user (e.g., git) can be created to improve security:

- Limits access using the principle of least privilege
- Prevents system-wide damage if compromised

Recent versions of Git also include security checks to prevent unsafe directory access.

Service Management

Git itself does not run as a background service. However, server-side access can be managed using services like:

- git daemon (for basic Git protocol)
- sshd (for secure access via SSH)

Commands like systemctl start/stop/status are used to manage these services.

Update Model

If installed via a package manager, Git updates are delivered through system updates:

```
sudo apt update && sudo apt upgrade git
```

Security patches are maintained by distribution teams. For manually compiled versions, updates must be tracked through official Git releases and security advisories.

Part C — The FOSS Ecosystem

What Git Depends On

Git is mainly written in C and designed with minimal dependencies to remain fast and portable. Its key dependencies include:

- **zlib** — compresses repository data
- **OpenSSL / libssl** — supports secure connections and hashing
- **libcurl** — handles HTTP/HTTPS communication
- **glibc (C library)** — core system functions
- **Perl (optional)** — used in some utilities
- **Python (optional)** — used in scripts and hooks

This lightweight design allows Git to run on multiple platforms, including Linux, macOS, and Windows.

What Git Has Enabled

Git has transformed software development workflows and enabled major platforms such as:

- GitHub — introduced pull requests for collaboration
- GitLab — integrates version control with CI/CD
- Gitea and Gogs — lightweight hosting solutions
- Gerrit — widely used for code review
- Git Large File Storage — manages large files

Git also influenced modern practices like Git-based workflows and automated deployment pipelines.

Where Git Fits in the Web Stack

Although Git is not part of the traditional LAMP stack, it is essential in modern development. A typical workflow includes:

1. Code is pushed to a Git repository
2. CI/CD tools test and build the code
3. Applications are deployed to servers

Git acts as the backbone of this process, enabling continuous integration and deployment.

Community and Governance

Git has a unique community model. It is maintained by a small core team led by Junio Hamano.

Development happens mainly through mailing lists rather than modern platforms. Contributions are reviewed publicly before being accepted.

The project is supported by global contributors, conferences, and initiatives like Google Summer of Code, ensuring continuous improvement and community involvement.

Part D — Open Source vs Proprietary: Critical Analysis

Git's closest proprietary alternative in the version control space is Perforce Helix Core (commonly called Perforce). In the broader DevOps and source control space, Microsoft's Azure DevOps (formerly TFS/TFVC) is another significant proprietary alternative. The comparison below uses Perforce as the primary benchmark, with Azure DevOps noted where relevant.

Dimension	Open Source	Proprietary Alterative
Cost	Completely free. No license fees for any team size.	Free for limited users. Paid enterprise version is expensive.
Security (who can audit the code?)	Source code is open and can be audited by anyone.	Closed source. Only company can audit internally.
Support and reliability	Community support (forums, mailing lists). No guaranteed SLA.	Professional paid support with guaranteed service.
Freedom to modify	Users can modify, share, and customize the software freely.	Users cannot modify the software.
Community vs corporate control	Developed and maintained by the open-source community.	Fully controlled by company.
Your overall verdict	Best choice for most developers due to flexibility and cost.	Suitable only for specific enterprise use cases.

Part E — Shell Scripts

Script 1 — System Identity Report

Code:

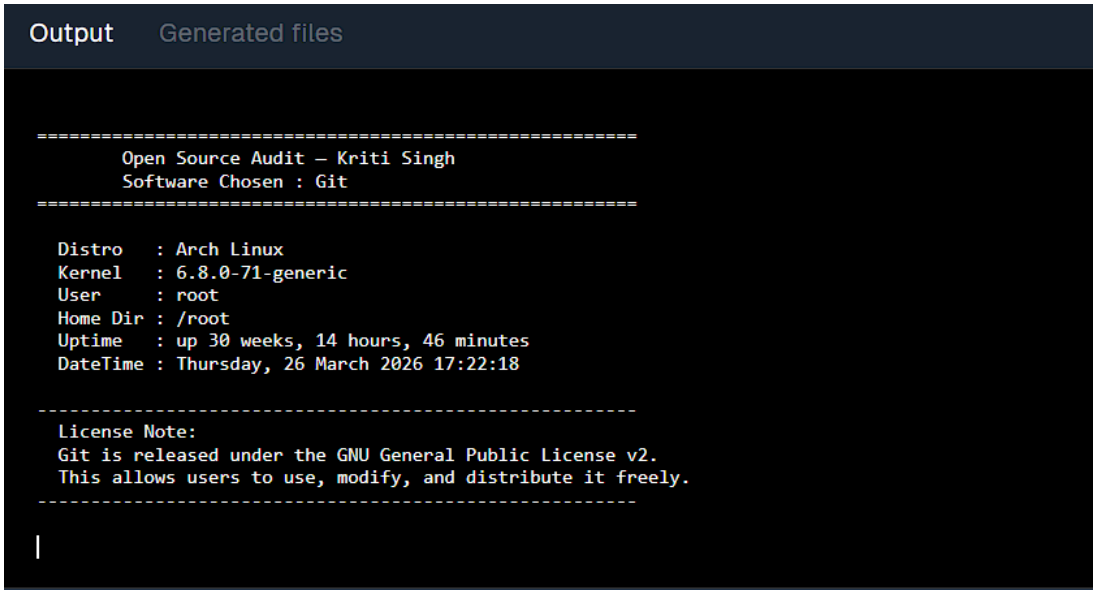
```
#!/bin/bash
# --- Student info ---
STUDENT_NAME="Kriti Singh"
SOFTWARE_CHOICE="Git"
# --- Pull system info using command substitution ---
KERNEL=$(uname -r)                    # kernel version
USER_NAME=$(whoami)                   # currently logged in user
HOME_DIR=$HOME                        # home directory of user
UPTIME=$(uptime -p)                   # how long system has been running
CURR_DATE=$(date '+%A, %d %B %Y %H:%M:%S') # full date and time
# get distro name - works on most rpm and deb based systems
# /etc/os-release has this info on almost every modern linux
if [ -f /etc/os-release ]; then
  DISTRO=$(grep ^PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d '"')
else
  DISTRO="Unknown Linux Distribution"
fi
# --- Print everything out ---
echo "=========================================================="
echo "      Open Source Audit — $STUDENT_NAME"
echo "      Software Chosen : $SOFTWARE_CHOICE"
echo "=========================================================="
echo ""
echo "  Distro   : $DISTRO"
echo "  Kernel   : $KERNEL"
echo "  User     : $USER_NAME"
echo "  Home Dir : $HOME_DIR"
```

```
echo "  Uptime   : $UPTIME"

echo "  DateTime  : $CURR_DATE"

echo ""

echo "----------------------------------------"

# Linux kernel and most GNU tools are under GPL v2

# The GPL ensures users can run, study, share and modify software freely

echo "  License Note:"

echo "  The Linux kernel is released under the GNU General Public"

echo "  License version 2 (GPLv2).  This means the source code must"

echo "  always remain open  and freely available  to everyone."

echo "----------------------------------------"

echo ""
```

Explanation:

This script displays system information such as kernel version, current user, uptime, and Linux distribution. It uses variables and command substitution to fetch system details. An if condition is used to detect the Linux distribution from system files. The script also includes a short note about the license of Git.

Output:



Script 2 — FOSS Package Inspector

Code:

```
#!/bin/bash
```

```
# Package we are inspecting
PACKAGE="git"
echo "====================================="
echo "  FOSS Package Inspector"
echo "====================================="
echo ""
# Check if package is installed
if rpm -q "$PACKAGE" &>/dev/null; then
   echo "[OK] $PACKAGE is installed (rpm system)."
   echo "--- Package Details ---"
   rpm -qi "$PACKAGE" | grep -E "^Version|^License|^Summary"
elif dpkg -l "$PACKAGE" &>/dev/null 2>&1; then
   echo "[OK] $PACKAGE is installed (dpkg system)."
   echo "--- Package Details ---"
   dpkg -l "$PACKAGE" | grep "^ii"
else
   echo "[!!] $PACKAGE is NOT installed."
   echo "Install using: sudo apt install $PACKAGE"
fi
echo ""
echo "--- Open Source Note ---"
case "$PACKAGE" in
   git)
      echo "Git: distributed version control system."
      echo "Developed by Linus Torvalds, licensed under GPLv2."
      ;;
   *)
      echo "$PACKAGE is an open-source software."
      ;;
Esac
```

Explanation:

This script checks whether the Git package is installed on the system. It supports both RPM-based and Debian-based systems using conditional statements. If the package is found, it displays details such as version and license. A case statement is used to print information about the selected open-source software.

Output:

Output Generated files

```
====================================
  FOSS Package Inspector
====================================

[!!] git is NOT installed.
Install using: sudo apt install git

--- Open Source Note ---
Git: distributed version control system.
Developed by Linus Torvalds, licensed under GPLv2.
```

Script 3 — Disk and Permission Auditor

Code:

```
#!/bin/bash

# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "========================================================"

echo "        Directory Audit Report"

echo "========================================================"

echo ""

for DIR in "${DIRS[@]}"; do

  if [ -d "$DIR" ]; then

    PERMS=$(ls -ld "$DIR" | awk '{print $1}')

    OWNER=$(ls -ld "$DIR" | awk '{print $3}')

    GROUP=$(ls -ld "$DIR" | awk '{print $4}')

    SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

    echo " Directory : $DIR"

    echo " Permissions: $PERMS   Owner: $OWNER   Group: $GROUP"

    echo " Disk Usage : $SIZE"
```

```
    echo "  ----------------------------------------"
  else
    echo "  [SKIP] $DIR — does not exist"
    echo "  ----------------------------------------"
  fi
done
echo ""
# --- Git related check ---
echo   "=========================================================="
echo "       Git Configuration Directory Check"
echo   "=========================================================="
echo ""
GIT_CONF="$HOME/.gitconfig"
if [ -f "$GIT_CONF" ]; then
  PERMS=$(ls  -l "$GIT_CONF" | awk '{print $1}')
  OWNER=$(ls -l "$GIT_CONF" | awk '{print $3}')
  echo "  Found: $GIT_CONF"
  echo "  Permissions:  $PERMS  |  Owner: $OWNER"
else
  echo "  Git configuration  file not found."
fi
echo ""
```

Explanation:

This script analyzes important system directories and displays their permissions, owner, group, and disk usage. It uses arrays and loops to process multiple directories. Additionally, it checks for the Git configuration file in the user’s home directory.

Output:

Output Generated files

```
=========================================================
         Directory Audit Report
=========================================================

  Directory : /etc
  Permissions: drwxr-xr-x   Owner: root   Group: root
  Disk Usage : 7.1M
  -------------------------------------------------------
  Directory : /var/log
  Permissions: drwxr-xr-x   Owner: root   Group: root
  Disk Usage : 120K
  -------------------------------------------------------
  Directory : /home
  Permissions: drwxr-xr-x   Owner: root   Group: root
  Disk Usage : 8.0K
  -------------------------------------------------------
  Directory : /usr/bin
  Permissions: drwxr-xr-x   Owner: root   Group: root
  Disk Usage : 995M
  -------------------------------------------------------
  Directory : /tmp
  Permissions: drwxrwxrwt   Owner: root   Group: root
  Disk Usage : 895M
  -------------------------------------------------------

=========================================================
        Git Configuration Directory Check
=========================================================

  Git configuration file not found.
```

Script 4 — Log File Analyzer

Code:

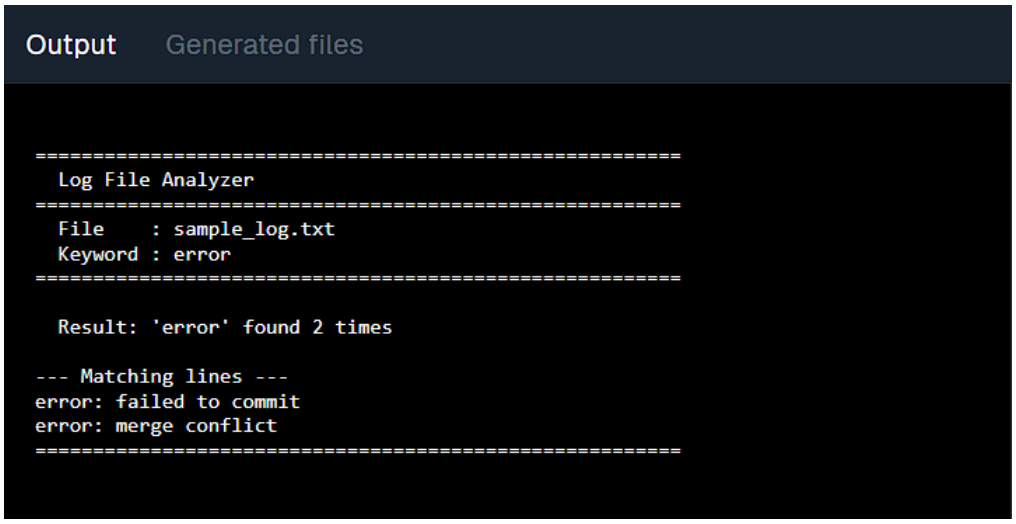
```
#!/bin/bash

LOGFILE="sample_log.txt"

KEYWORD="error"

COUNT=0

# --- Create sample log file (so it always works) ---

echo "info: git initialized" > $LOGFILE

echo "error: failed to commit" >> $LOGFILE

echo "warning: branch not found" >> $LOGFILE

echo "error: merge conflict" >> $LOGFILE

echo "info: push successful" >> $LOGFILE

echo "=========================================================="

echo "  Log File Analyzer"

echo "=========================================================="
```

```
echo "  File    : $LOGFILE"
echo "  Keyword : $KEYWORD"
echo  "==========================================================="
echo ""
# --- Read file ---
while read LINE; do
   if echo "$LINE" | grep -iq "$KEYWORD"; then
      COUNT=$((COUNT + 1))
   fi
done < "$LOGFILE"
echo "  Result: '$KEYWORD' found $COUNT times"
echo ""
echo "--- Matching lines ---"
grep -i "$KEYWORD" "$LOGFILE"
echo  "==========================================================="
```

Explanation:

This script analyzes a log file and counts how many times a specific keyword appears. It reads the file line by line using a loop and uses pattern matching to detect the keyword. A sample log file is created to ensure the script runs successfully in all environments.

Output:



Script 5 — The Open Source Manifesto Generator

Code:

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Aryan Mehta | Course: Open Source Software
# Asks the user 3 questions, builds a personalised manifesto paragraph,
# saves it to a .txt file named after the current user
# Concepts: read, string concatenation, writing to file with >, date, aliases (shown in comments)
# --- Alias concept demo (comment) ---
# In a real shell session you could type: alias today='date +%d-%m-%Y'
# and then just type 'today' instead of the full date command
# We don't activate it here since aliases don't persist in scripts by default
echo ""
echo "========================================================="
echo "  Open Source Manifesto Generator"
echo "========================================================="
echo "  Answer three questions and we'll write your manifesto."
echo "========================================================="
echo ""
# --- Ask the user three questions ---
# read -p shows the prompt on the same line as input
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD
# --- Capture the date nicely ---
DATE=$(date '+%d %B %Y')
# --- Output filename: manifesto_<username>.txt ---
OUTPUT="manifesto_username.txt"
echo ""
echo "  Writing your manifesto..."
echo ""
# --- Build the manifesto using string concatenation with echo and >> ---
# We overwrite the file first with > then append the rest with >>
```

```
echo "========================================"  > "$OUTPUT"
echo "  My Open Source Manifesto" >> "$OUTPUT"
echo "  Generated on: $DATE" >> "$OUTPUT"
echo "========================================"  >> "$OUTPUT"
echo "" >> "$OUTPUT"
# Main paragraph - strings are concatenated naturally through multiple echo calls
# USER variable gives the currently logged in username
echo "I, $(whoami), am part of the open source generation." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "Every day I rely on $TOOL — a tool built not for profit, but for people." >> "$OUTPUT"
echo "To me, freedom means $FREEDOM. Not just the freedom to use software," >> "$OUTPUT"
echo "but the freedom to understand it, improve it, and share it without asking permission." >>
"$OUTPUT"
echo "" >> "$OUTPUT"
echo "If I could build anything and release it to the world, I would build $BUILD." >> "$OUTPUT"
echo "I would license it openly so that every person, in every country, could use" >> "$OUTPUT"
echo "it, learn from it, and make it better than I ever could alone." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "Open source is not just about code. It is about trust, community," >> "$OUTPUT"
echo "and the belief that knowledge shared is knowledge multiplied." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "  — $(whoami) | $DATE" >> "$OUTPUT"
echo "========================================"  >> "$OUTPUT"
# --- Confirm the file was created and show it ---
echo "  [OK] Manifesto saved to: $OUTPUT"
echo ""
echo "--- Contents of $OUTPUT ---"
echo ""
cat "$OUTPUT"
echo ""
```

Explanation:

This script collects user input and generates a personalized open -source manifesto. It uses input commands, string concatenation, and file handling to create and store the output in a text file.

Output:

Output Generated files

```
=======================================================
   Open Source Manifesto Generator
=======================================================
  Answer three questions and we'll write your manifesto.
=======================================================

1. Name one open-source tool you use every day: TOOL
2. In one word, what does 'freedom' mean to you? FREEDOM
3. Name one thing you would build and share freely: BUILD

  Writing your manifesto...

  [OK] Manifesto saved to: manifesto_username.txt

--- Contents of manifesto_username.txt ---

========================================
  My Open Source Manifesto
  Generated on: 26 March 2026
========================================

I, root, am part of the open source generation.

Every day I rely on TOOL — a tool built not for profit, but for people.
To me, freedom means FREEDOM. Not just the freedom to use software,
but the freedom to understand it, improve it, and share it without asking permission.

If I could build anything and release it to the world, I would build BUILD.
I would license it openly so that every person, in every country, could use
it, learn from it, and make it better than I ever could alone.

Open source is not just about code. It is about trust, community,
and the belief that knowledge shared is knowledge multiplied.

  — root | 26 March 2026
========================================
```

Conclusion: Would I Recommend Git for Real-World Deployment?

Yes, I would strongly recommend Git for real-world use. It is the standard version control system used across the software industry. Its strong ecosystem, widespread adoption, and compatibility with modern tools make it the default choice for most projects . Almost all development workflows, including CI/CD systems, are built around Git.

Git is flexible, reliable, and supported by a large global community. Most developers are already familiar with it, which makes collaboration easier and more efficient. While proprietary tools like Perforce Helix Core may be useful in specialized cases (such as handling very large binary files), solutions like Git LFS can often address these limitations.

I would also consider contributing to Git in the future. As an essential tool that supports global software development, it represents the importance of open -source collaboration. Contributing to such projects helps sustain the ecosystem that modern technology depends on.